

Registro semana 7 mobile

Nome: Arthur Saraiva de França 3B.

Objetivo da atividade

O objetivo desta atividade foi aplicar técnicas de refatoração em um código Kotlin, melhorando sua organização, legibilidade e manutenção, sem alterar seu funcionamento.

Análise do código original

O código fornecido funciona, porém apresenta alguns problemas:

- Uso de strings para status (pode causar erro)
 - Código pouco organizado
 - Difícil de manter e expandir
 - Lógica toda concentrada em um único método
 - Uso de Thread.sleep() diretamente
-

Código original

```
class PedidoProcessor {  
    fun processarPedido(pedido: Pedido) {  
        when (pedido.status) {  
            "novo" -> {  
                println("Pedido em processamento: ${pedido.id}")  
                Thread.sleep(2000)  
            }  
            "processado" -> {  
                println("Pedido já processado: ${pedido.id}")  
            }  
        }  
    }  
}
```

```
        else -> {
            println("Status desconhecido do pedido: ${pedido.id}")
        }
    }
}
}
```

Código refatorado

```
interface PedidoStrategy {
    fun processar(pedido: Pedido)
}

class NovoPedidoStrategy : PedidoStrategy {
    override fun processar(pedido: Pedido) {
        println("Pedido em processamento: ${pedido.id}")
    }
}

class PedidoProcessadoStrategy : PedidoStrategy {
    override fun processar(pedido: Pedido) {
        println("Pedido já processado: ${pedido.id}")
    }
}

class PedidoDesconhecidoStrategy : PedidoStrategy {
    override fun processar(pedido: Pedido) {
        println("Status desconhecido do pedido: ${pedido.id}")
    }
}

class PedidoProcessor {

    fun processarPedido(pedido: Pedido) {
        val strategy = when (pedido.status) {
```

```
"novo" -> NovoPedidoStrategy()
"processado" -> PedidoProcessadoStrategy()
else -> PedidoDesconhecidoStrategy()
}

strategy.processar(pedido)
}
}
```

O que foi melhorado

- Separação da lógica em várias classes
 - Código mais organizado
 - Mais fácil de entender
 - Facilita adicionar novos tipos de pedido
-

Respostas das perguntas

1. Quais problemas você identificou no código original?

- Uso de strings para controle de status
 - Código concentrado em um único método
 - Difícil manutenção
 - Baixa legibilidade
-

2. Como a refatoração melhorou o código?

A refatoração deixou o código mais organizado, separando responsabilidades em diferentes classes. Isso facilita a leitura e entendimento, além de tornar o código mais limpo.

3. Quais padrões de design foram aplicados e por quê?

Foi utilizado o padrão Strategy, pois permite separar diferentes comportamentos (tipos de pedido) em classes diferentes, facilitando a manutenção e expansão do sistema.

4. Impacto na manutenção e escalabilidade

As mudanças facilitam a manutenção, pois cada tipo de pedido pode ser alterado separadamente. Também melhora a escalabilidade, pois novos tipos podem ser adicionados sem alterar o código principal.

5. Boas práticas aplicadas

- Separação de responsabilidades (SRP)
 - Uso de interfaces
 - Código mais limpo e organizado
 - Redução de complexidade
-

Conclusão

A refatoração tornou o código mais organizado, legível e fácil de manter. A aplicação do padrão Strategy ajudou a melhorar a estrutura do sistema, deixando ele mais preparado para futuras mudanças.